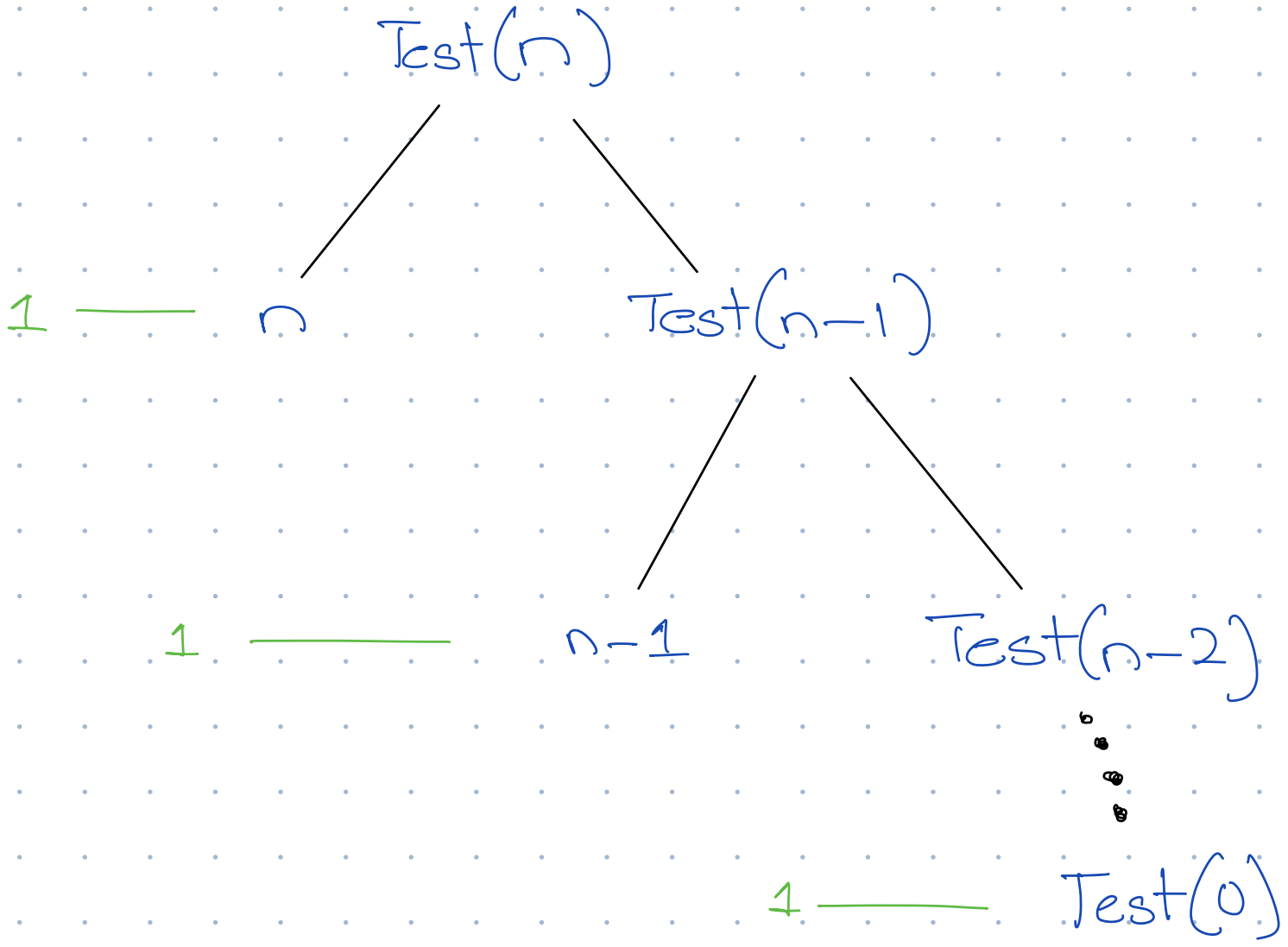
Recursive Functions.

```
Void Test (int n)
{
    if (n > 0)
    {
      cout << n; ← (1)
      Test (n-1).
    }
}.
```



$c * n$

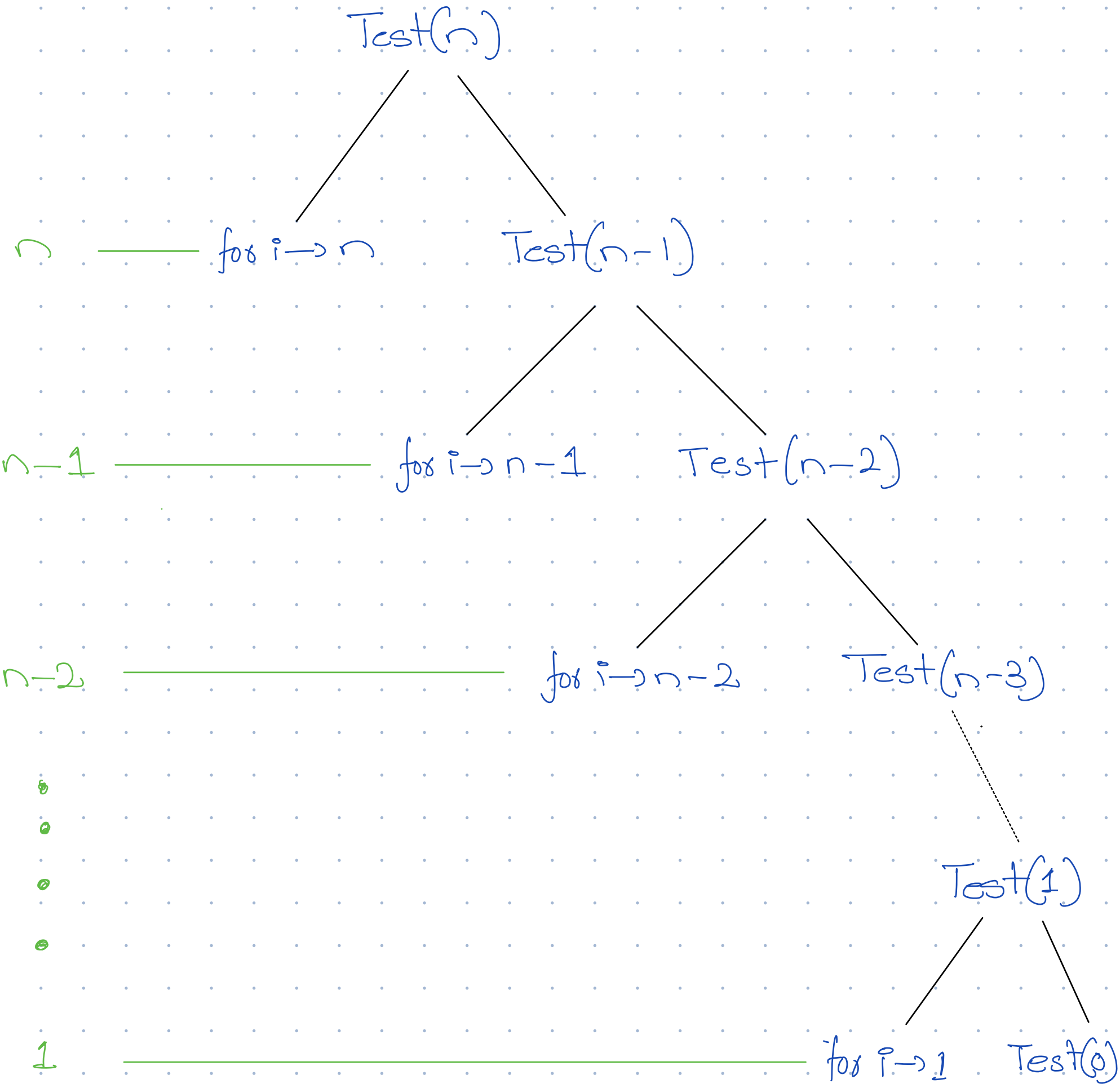
$O(n)$

```
void Test(int n) {
    if (n>0) {                          ——— 1
        for(int i=0; i<n; i++) {        ——— n+1
            printf(n);                  ——— n
        }
    }
}
```

Test(n-1); ——— 1

}

{



$$n + n-1 + n-2 + \dots + 1 = \frac{n(n+1)}{2}$$

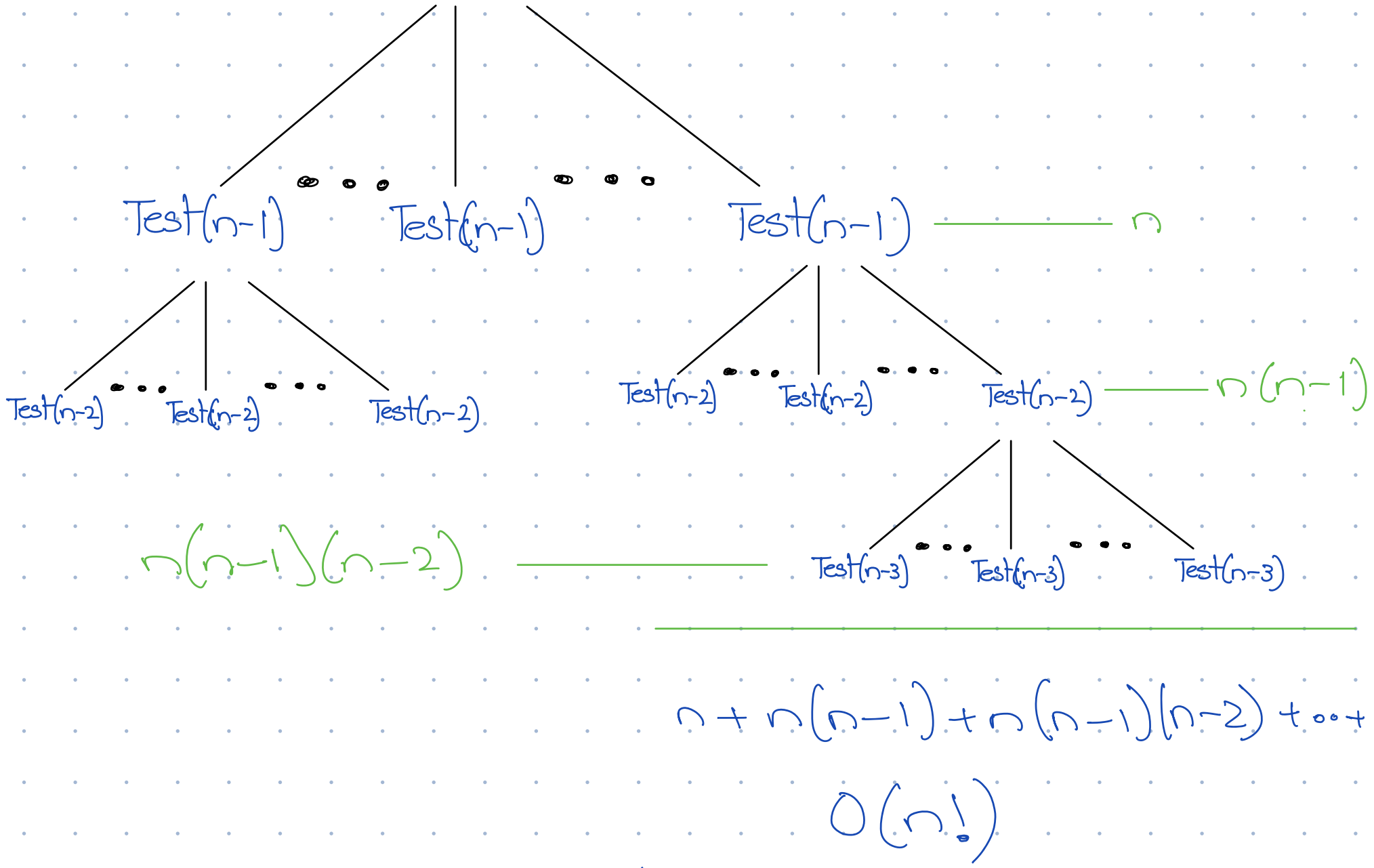
$$= O(n^2)$$

NOTE: The actual complexity of each recursive call is $2k+3$. Thus, for each branch we would have $2n+3$, $2(n-1)+3$ and so on.

for simplicity, we consider only the dominant term i.e.

```
void Test(int n) {
    if (n>0) {                                  ——— 1
        for(int i=0; i<n; i++) {                ——— n+1
            Test(n-1);                          ——— n
        }
    }
}
```

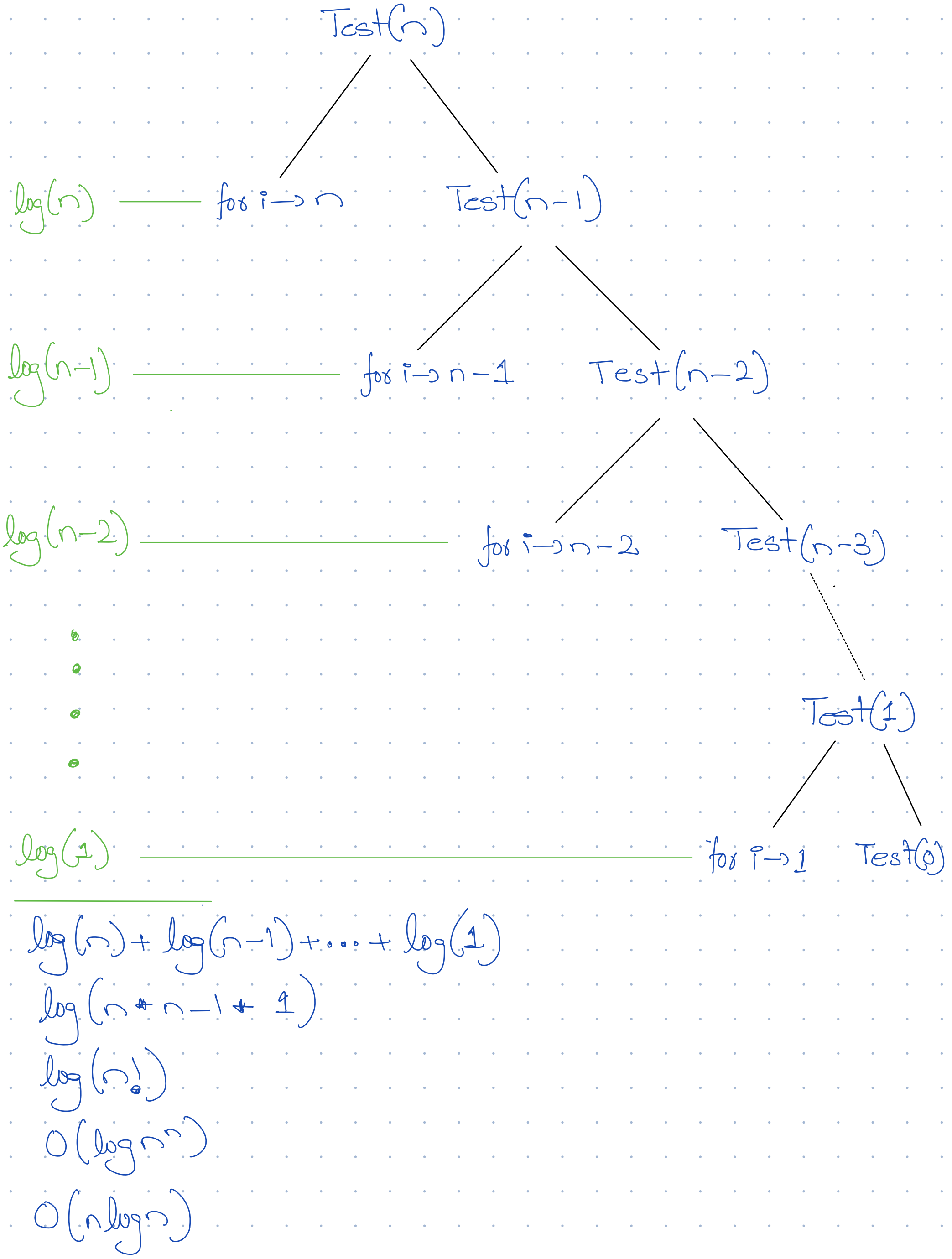
Test(n)



$n + n(n-1) + n(n-1)(n-2) + \cdots + n!$

$O(n!)$

```
void Test(int n) {
    if (n>0) {                                  ——— 1
        for(int i=0; i<n; i*=2) {               ——— log(n)+1
            printf(n);                          ——— log(n)
        }
        Test(n-1);                              ——— 1
    }
}
```


Test(n)
log(n)
for i -> n
Test(n-1)
log(n-1)
for i -> n-1
Test(n-2)
log(n-2)
for i -> n-2
Test(n-3)
Test(1)
log(1)
for i -> 1
Test(0)
log(n) + log(n-1) + ... + log(1)
log(n * n-1 * 1)
log(n!)
O(log n^n)
O(n log n)

1. **Divide (Branching):** You break the main problem down into its subcalls, drawing them as children of the parent node. This builds the shape of your tree.
2. **Find Node Work (The Cost):** Inside each specific node, you write down *only* the extra work done by that specific function call (like the $O(n/4)$ work done by the helper function at depth 2).
3. **Sum the Levels (Horizontal Math):** You look at one horizontal row of the tree at a time and add up the work inside all those nodes. This tells you how much total processing power that specific depth of recursion requires.
4. **Sum the Tree (Vertical Math):** You take the totals from every level and add them all together to get your final, big-O time complexity.